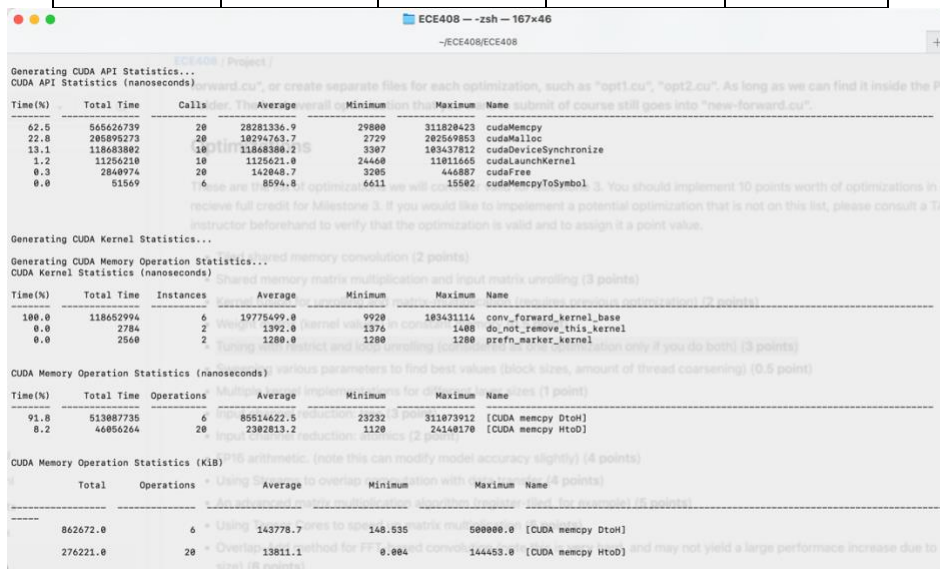


Name: Han Chen
NetID: Hanc7
Section: Zj1

ECE 408/CS483 Milestone 3 Report

0. List Op Times, whole program execution time, and accuracy for batch size of 100, 1k, and 5k images from your basic forward convolution kernel in milestone 2. This will act as your baseline this milestone. Note: **Do not** use batch size of 10k when you profile in `--queue rai_amd64_exclusive`. We have limited resources, so any tasks longer than 3 minutes will be killed. Your baseline M2 implementation should comfortably finish in 3 minutes with a batch size of 5k (About 1m35 seconds, with nv-nsight).

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.318185 ms	2.08727 ms	0m0.193s	0.86
1000	3.04541 ms	20.5922 ms	0m0.385s	0.886
5000	15.6676 ms	101.485 ms	0m1.084s	0.871



The screenshot displays the NVIDIA NSIGHT Systems interface for a project named 'ECE408'. It shows two main sections: 'Generating CUDA API Statistics...' and 'Generating CUDA Kernel Statistics...'. The API statistics table lists various CUDA calls with their total time, number of calls, average, minimum, and maximum durations. The kernel statistics table lists specific kernels with their total time, number of instances, average, minimum, and maximum durations. The memory operation statistics table lists memory operations with their total time, number of operations, average, minimum, and maximum durations. The kernel statistics table also includes a 'Name' column with the kernel name.

Time(%)	Total Time	Calls	Average	Minimum	Maximum	Name
42.5	565626739	20	28281336.9	29800	311820423	cudaMemcpy
22.8	205895273	20	10294763.7	2729	202569853	cudaMalloc
13.1	118683802	10	11868380.2	3307	103437812	cudaDeviceSynchronize
1.2	11256210	10	1125621.0	24460	11811665	cudaLaunchKernel
0.3	2640974	20	142048.7	3205	444887	cudaFree
0.0	51569	6	8594.8	6611	15582	cudaMemcpyToSymbol

Time(%)	Total Time	Instances	Average	Minimum	Maximum	Name
100.0	118652994	6	19775499.0	9920	103431114	conv_forward_kernel_base
0.0	2784	2	1392.0	1376	1408	do_not_remove_this_kernel
0.0	2660	2	1280.0	1280	1280	prefn_marker_kernel

Time(%)	Total Time	Operations	Average	Minimum	Maximum	Name
91.8	513887735	6	85514622.5	23232	311873912	[CUDA memcpy DtoH]
8.2	46856264	20	2382813.2	1120	24148170	[CUDA memcpy HtoD]

Total	Operations	Average	Minimum	Maximum	Name
862672.0	6	143778.7	148.535	500000.0	[CUDA memcpy DtoH]
276221.0	20	13811.1	0.004	144453.0	[CUDA memcpy HtoD]

1. **Optimization 1: Weight matrix (kernel values) in constant memory (0.5 point)**

- a. Which optimization did you choose to implement and why did you choose that optimization technique.

Weight matrix (kernel values) in constant memory, because this can save the times that we retrieve value from global memory.

- b. How does the optimization work? Did you think the optimization would increase performance of the forward convolution? Why? Does the optimization synergize with any of your previous optimizations?

This optimization replaces reading from global memory to constant memory. Yes, this is because it reduces the times that you use to retrieve value from global memory. This is the first optimization, but I am sure that it can synergize with any other optimization as other optimization will not change the usage of mask array.

- c. List the Op Times, whole program execution time, and accuracy for batch size of 100, 1k, and 5k images using this optimization (including any previous optimizations also used).

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.281437 ms	1.86507 ms	0m0.225s	0.86
1000	2.67664 ms	18.412 ms	0m0.396s	0.886
5000	13.3038 ms	92.1523 ms	0m0.916s	0.871

- d. Was implementing this optimization successful in improving performance? Why or why not? Include profiling results from *nsys* and *Nsight-Compute* to justify your answer, directly comparing to your baseline (or the previous optimization this one is built off of).

```

Generating CUDA API Statistics...
CUDA API Statistics (nanoseconds)

Time(%)    Total Time    Calls        Average        Minimum        Maximum    Name
-----
62.8        500003631        20        25000181.6        20056        265895917    cudaMemcpy
21.7        172451281        20        8622564.1        2776        169451848    cudaMalloc
13.2        106188242        10        10618824.2        3839        91818731    cudaDeviceSynchronize
2.0         15706467        10        1570646.7        15730        15764823    cudaLaunchKernel
0.3         2271183        20        113559.1        3033        411776    cudaFree
0.0         44029        6        7438.2        6190        10430    cudaMemcpyToSymbol

Generating CUDA Kernel Statistics...

Generating CUDA Memory Operation Statistics...
CUDA Kernel Statistics (nanoseconds)

Time(%)    Total Time    Instances    Average        Minimum        Maximum    Name
-----
100.0      106160206        4        17526701.0        10176        91815314    conv_forward_kernel_constant
0.0         2720        2        1360.0        1344        1376    do_not_remove_this_kernel
0.0         2660        2        1280.0        1216        1344    prefn_marker_kernel

CUDA Memory Operation Statistics (nanoseconds)

Time(%)    Total Time    Operations    Average        Minimum        Maximum    Name
-----
92.4       456489934        6        76081655.7        12288        265213508    [CUDA memcpy DtoH]
7.6        37862440        20        1878122.0        1120        19363151    [CUDA memcpy HtoD]

CUDA Memory Operation Statistics (KiB)

Total      Operations    Average        Minimum        Maximum    Name
-----
-----

```

Details
Launches: 0 - 120 - conv_forward_kernel_2 - Add Baseline - Apply Filter

Details
120 - conv_forward_kernel_constant [1, 196, 11x16, 16, 1]
Time: 31.14 seconds
Reqs: 30 GPU: TITAN V SM Frequency: 1.13 cycl/smaxcon: CC: 7.0 Process: [556] m3

Save as Image

GPU Speed of Light

High-level overview of GPU utilization for compute and memory resources of the GPU. For each unit, the Speed of Light (SOL) reports the achieved percentage of utilization with respect to the theoretical maximum. High-level overview of the utilization for compute and memory resources of the GPU presented as a roffline chart.

SM [N]	25.38	Duration (second)	31.14
SOL Memory [N]	75.26	Elapsed Cycles (cycle)	31,357
SOL L1/TEX Cache [N]	88.48	SM Active Cycles (cycle)	39,855.56
SOL L2 Cache [N]	15.80	SM Frequency (cycles/second)	1.13
SOL SMW [N]	3.33	SMW Frequency (cycle/second)	119.44

GPU Utilization

SOL SM Breakdown

SOL SM: Issue Active [N]	25.30
SOL SM: Not Executed [N]	25.38
SOL SM: Pipe Free Cycles Active [N]	21.23
SOL SM: Pipe Ali Cycles Active [N]	17.07
SOL SM: Not Executed Pipe Ali [N]	20.10
SOL SOC: Request Cycles Active [N]	9.19
SOL SM: Not Executed Pipe Lru [N]	7.16
SOL SM: Not Executed [N]	6.13
SOL SM: Init Executed Pipe Chu Prod On Any [N]	6.01
SOL SM: Mix P0 Read Cycles Active [N]	5.53
SOL SM: Mix02 Writeback Active [N]	5.46
SOL SM: Mix P0 Write Cycles Active [N]	4.76
SOL SM: Not Executed Pipe Xu [N]	3.90
SOL SM: Not Executed Pipe P0 [N]	0
SOL SM: Init Executed Pipe lru [N]	0
SOL SM: Not Executed Pipe P04 [N]	0
SOL SM: Pipe P04 Cycles Active [N]	0
SOL SM: Pipe Shared Cycles Active [N]	0
SOL SM: Pipe Tensor Cycles Active [N]	0

SOL Memory Breakdown

SOL L1: Data Pipe Lru Waveforms [N]	77.34
SOL L2: Lru Waveform Active [N]	25.93
SOL L2: Mix02 Cycles Active [N]	15.99
SOL L2: T7 Sectors [N]	10.49
SOL L2: T7 Ray Requests [N]	10.46
SOL L1: M LruWaveform Ray Cycles Active [N]	6.61
SOL L1: Lru Requests [N]	7.16
SOL L2: LruWaveform Cycles Active [N]	6.03
SOL L1: M Abv2Dlru Read Sectors [N]	3.37
SOL GPU: Dram Throughput [N]	1.31
SOL L2: D Sectors [N]	2.60
SOL L1: Data Bank Reads [N]	2.33
SOL L2: D Sectors Fill Device [N]	1.26
SOL L1: Data Bank Writes [N]	0
SOL L1: Train Sm2Tex Ray Cycles Active [N]	0.01
SOL L2: Ray Waveforms [N]	0.01
SOL L2: D Atomic Input Cycles Active [N]	0
SOL L2: D Sectors Fill Symptom [N]	0
SOL L1: No Writeback Active [N]	0
SOL L1: Data Pipe Lru Waveforms [N]	0

Floating Point Operations Roffline

Performance (GFlops)

10
1
0.1

Arithmetic Intensity (FLOP/Byte)

10
100
1,000

Recommendations

Bottleneck

Warning: Memory is more heavily utilized than Compute. Look at the [Memory Resource](#) report section to see where the memory system bottleneck is. Check memory replay (coalescing) metrics to make sure you're efficiently utilizing the data transferred. Also consider whether it's possible to do more work per memory access (kernel fusion) or whether there are values you can incorporate.

Roofline Analysis

The ratio of peak float (fp32) to double (fp64) performance on this device is 2.1. The kernel achieved 2% of this device's fp32 peak performance and 0% of its fp64 peak performance.

Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Warp Throughput (Inst/cycle)	1.87	SM Busy [N]	28.94
Executed Pipe Active (Inst/cycle)	1.15 <td>Issue Slots Busy [N]</td> <td>28.94</td>	Issue Slots Busy [N]	28.94
Issue Slot Active (Inst/cycle)	1.16		

Memory Workload Analysis

Detailed overview of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit. Deprecated UI elements for backwards compatibility.

Warp Throughput (Inst/cycle)	19.36	Mem Busy [N]	77.34
L1/TEX hit rate [N]	89.23 <td>Max Bandwidth [N]</td> <td>25.93</td>	Max Bandwidth [N]	25.93
L2 hit rate [N]	94.42 <td>Mem Pipes Busy [N]</td> <td>10.19</td>	Mem Pipes Busy [N]	10.19

Scheduler Statistics

Analysis of the activity of the scheduler issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions to. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the number of warps in the pool (Active Warps). Active warps that are not stalled (Stalled Warps) are ready to issue their next instruction. From the set of active warps the scheduler selects a single warp from which to issue one or more instructions (Issue Slots) with the next cycle warps. The issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates low utilization.

Active Warps Per Scheduler (warp)	10.84	No Eligible [N]	78.73
Eligible Warps Per Scheduler (warp)	0.78	One or More Eligible [N]	29.27
Issue Warps Per Scheduler	0.29		

Warp State Statistics

Analysis of the states in which all warps spend cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp preparation is required to issue the next instruction. The chart shows the average number of cycles spent to state per warp per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle, when executing a kernel with mixed busy and idle code, these metrics show the combined values.

Warp Cycles Per Issued Instruction (cycle)	34.31	Avg. Active Threads Per Warp	32.68
Warp Cycles Per Executed Instruction (cycle)	34.46	Avg. Not Predicted Off Threads Per Warp	28.90

Instruction Statistics

Statistics of the executed low-level assembly instructions (MASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that instructions (OpCode) and Executed instructions are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions (Inst)	2,845,626	Avg. Executed Instructions Per Scheduler (Inst)	8,992.58
Issued Instructions (Inst)	2,857,778	Avg. Issued Instructions Per Scheduler (Inst)	9,379.55

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	588	Registers Per Thread (register/thread)	39
Block Size	256	Static Shared Memory Per Block (byte/block)	0
Blocks (thread)	156.25	Dynamic Shared Memory Per Block (byte/block)	0
Waves Per SM	0.52	Driver Shared Memory Per Block (byte/block)	0
		Shared Memory Configuration Size (byte)	

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates high instruction overhead.

Theoretical Occupancy [N]	100	Block Limit Registers (block)	8
Theoretical Active Warps per SM (warp)	64	Block Limit Shared Mem (block)	32
Achieved Occupancy [N]	64.46	Block Limit Wave (block)	8

“constant memory” from lecture 7.

2. Optimization 2: Tiled shared memory convolution (2 points)

- a. Which optimization did you choose to implement and why did you choose that optimization technique.

Tiled shared memory convolution. I think this can reduce the number of times that I retrieve input values from global memory so that the performance will be better.

- b. How does the optimization work? Did you think the optimization would increase performance of the forward convolution? Why? Does the optimization synergize with any of your previous optimizations?

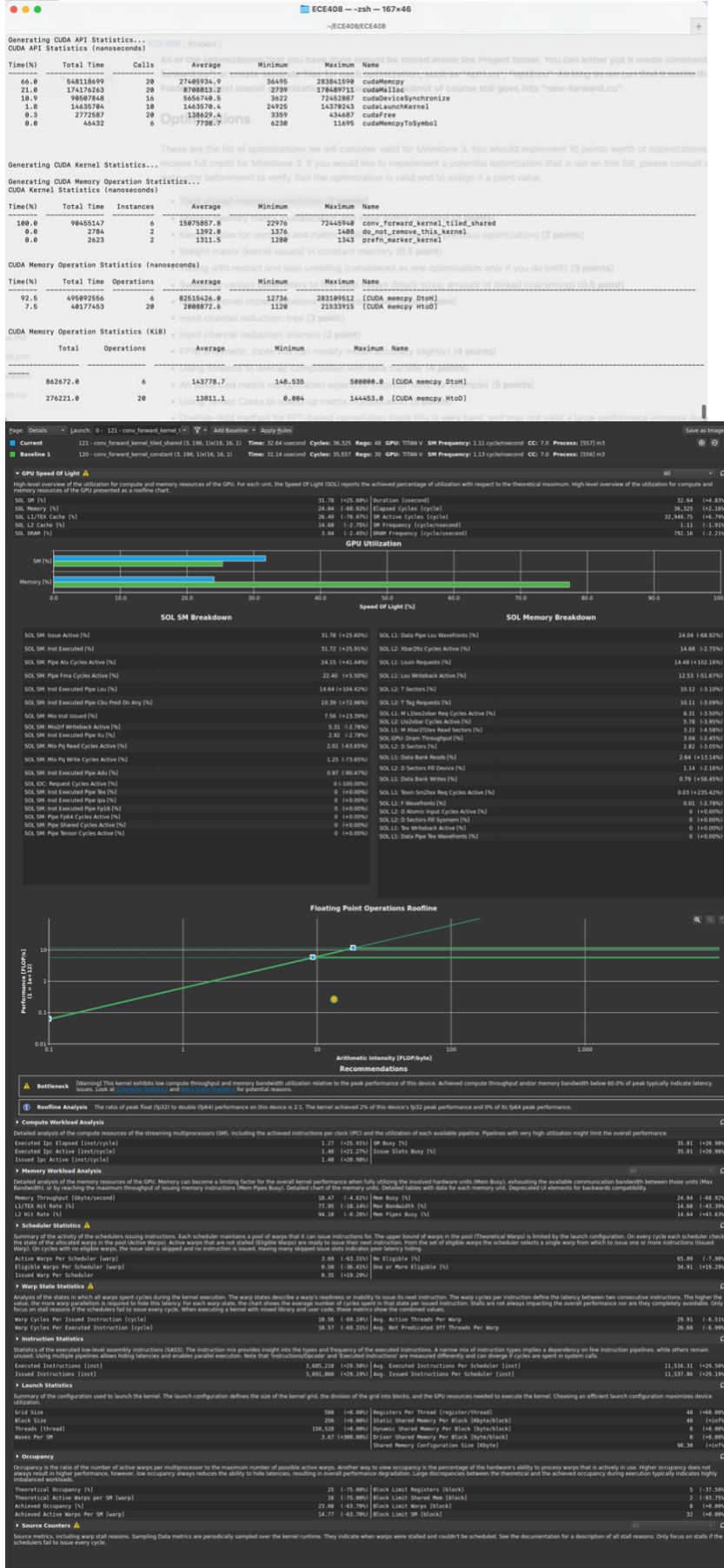
We will read a portion of input value and store into shared memory to reduce the times that I read from global memory so that we spend less time. Yes, this will synergize with previous optimization of storing the mask into constant memory.

- c. List the Op Times, whole program execution time, and accuracy for batch size of 100, 1k, and 5k images using this optimization (including any previous optimizations also used).

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.376543 ms	1.47727 ms	0m0.213s	0.86
1000	3.57791 ms	14.5086 ms	0m0.280s	0.886
5000	17.7052 ms	72.5626 ms	0m0.960s	0.871

- d. Was implementing this optimization successful in improving performance? Why or why not? Include profiling results from *nsys* and *Nsight-Compute* to justify your answer, directly comparing to your baseline (or the previous optimization this one is built off of).

Yes, it is successful. We will read a portion of input value and store into shared memory to reduce the times that I read from global memory so that we spend less time.



- e. What references did you use when implementing this technique?

The tiled convolution technique in lecture 8.

3. **Optimization 3: Tuning with restrict and loop unrolling (considered as one optimization only if you do both) (3 points)**

- a. Which optimization did you choose to implement and why did you choose that optimization technique.

Tuning with restrict and loop unrolling (considered as one optimization only if you do both). I choose this because it is easy to implement and has remarkable upgrade.

- b. How does the optimization work? Did you think the optimization would increase performance of the forward convolution? Why? Does the optimization synergize with any of your previous optimizations?

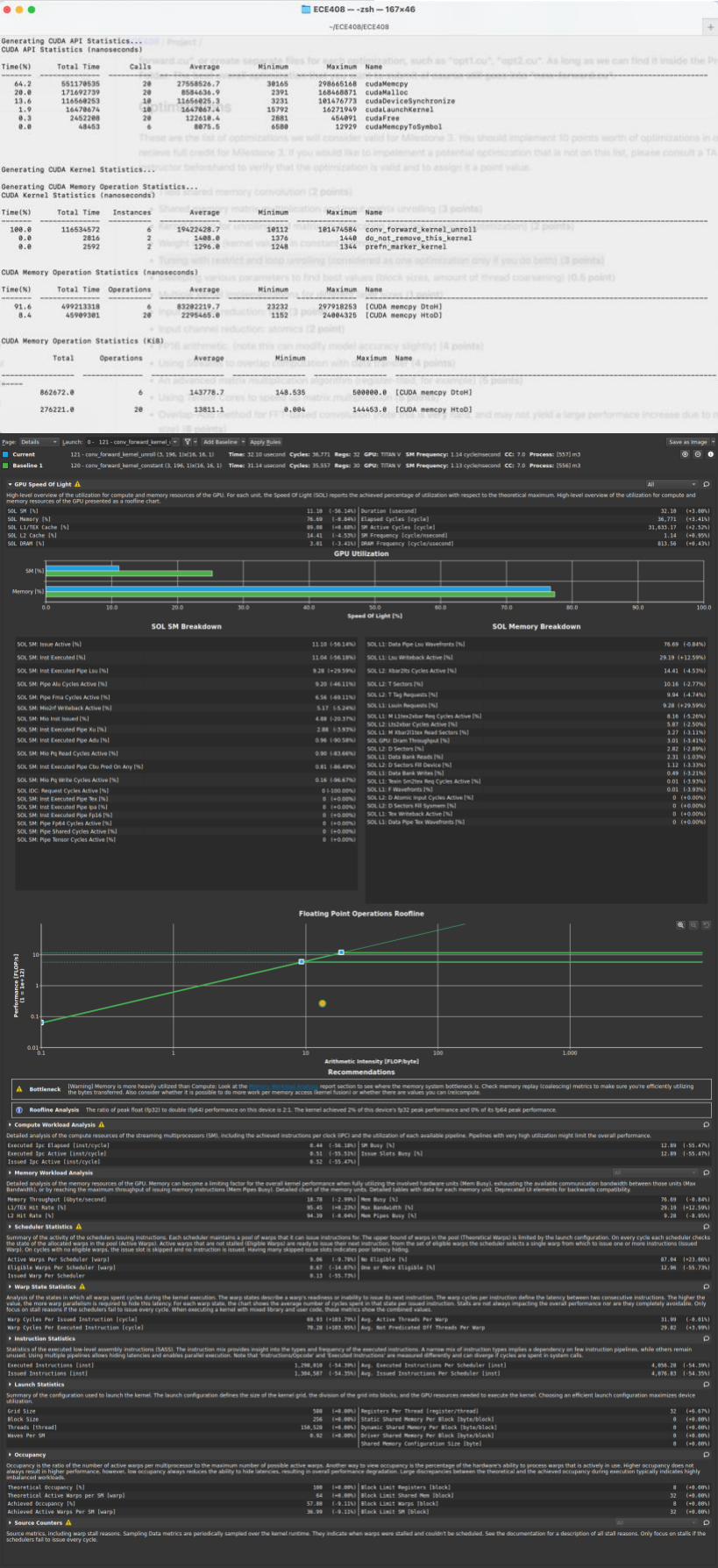
This optimization unrolls the calculation by loop, avoids using loop to calculate and directly adds up the sum of all three channels, so that we reduce the time in updating iterative number such as i. Yes it will increase the performance. And this optimization synergizes with all previous optimizations. Also, The __restrict__ keyword allows the compiler to make certain assumptions about the absence of pointer aliasing, which can lead to more aggressive optimizations.

- c. List the Op Times, whole program execution time, and accuracy for batch size of 100, 1k, and 5k images using this optimization (including any previous optimizations also used).

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.328119 ms	2.08456 ms	0m0.211s	0.86
1000	3.03235 ms	20.3188 ms	0m0.328s	0.886
5000	14.9039 ms	91.0078 ms	0m0.972s	0.871

- d. Was implementing this optimization successful in improving performance? Why or why not? Include profiling results from *nsys* and *Nsight-Compute* to justify your answer, directly comparing to your baseline (or the previous optimization this one is built off of).

Yes. This optimization avoids using loop to calculate and directly adds up the sum of all three channels, so that we reduce the time in updating iterative number such as i. Yes it will increase the performance.



- e. What references did you use when implementing this technique?

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#restrict>

4. Optimization 4: FP16 arithmetic. (note this can modify model accuracy slightly) (4 points)

- a. Which optimization did you choose to implement and why did you choose that optimization technique.

FP16 arithmetic. (note this can modify model accuracy slightly). I choose this because Mixed precision training offers significant computational speedup by performing operations in half-precision format, while storing minimal information in single-precision to retain as much information as possible in critical parts of the network.

- b. How does the optimization work? Did you think the optimization would increase performance of the forward convolution? Why? Does the optimization synergize with any of your previous optimizations?

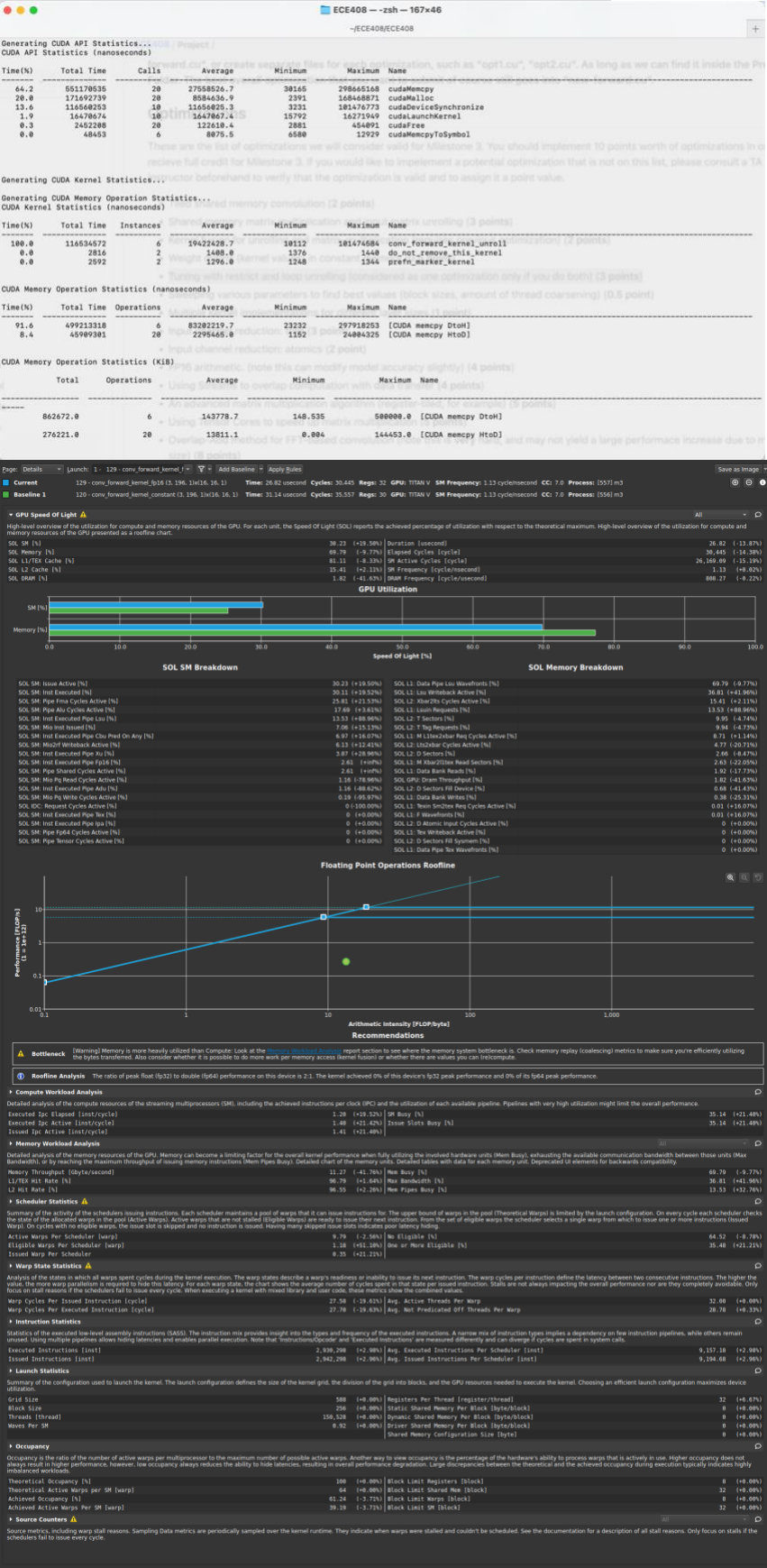
*I first use a kernel to cast device_in and mask into half type, then run base forward function (also change the type in function head to half type), and finally run another kernel to cast out from half type to float type.
I think it will increase the performance. And, The more float calculation, the better the performance is. This optimization can synergize will all previous optimization.*

- c. List the Op Times, whole program execution time, and accuracy for batch size of 100, 1k, and 5k images using this optimization (including any previous optimizations also used).

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.586076 ms	1.51916 ms	0m0.224s	0.86
1000	4.13122 ms	12.9303 ms	0m0.284s	0.887
5000	19.283 ms	63.3033 ms	0m0.764s	0.8712

- d. Was implementing this optimization successful in improving performance? Why or why not? Include profiling results from *nsys* and *Nsight-Compute* to justify your answer, directly comparing to your baseline (or the previous optimization this one is built off of).

Yes. This is because mixed precision training offers significant computational speedup by performing operations in half-precision format, while storing minimal information in single-precision to retain as much information as possible in critical parts of the network. Especially for large batch size, the optimization is better as there are more float computation that are switched to half computation.



- e. What references did you use when implementing this technique?

<https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>

5. **Optimization 5: Sweeping various parameters to find best values (block sizes, amount of thread coarsening) (0.5 point)**

- a. Which optimization did you choose to implement and why did you choose that optimization technique.

Sweeping various parameters to find best values (block sizes, amount of thread coarsening). I choose this because the coarsening and block size will affect the performance greatly.

- b. How does the optimization work? Did you think the optimization would increase performance of the forward convolution? Why? Does the optimization synergize with any of your previous optimizations?

I change different block size and record the final op time and compare the performance. I think maybe larger block size will have better performance in shared tile scenario as it stores more value and reduces more times of reading from global memory. And this optimization synergize with all previous optimizations. My record below is tested in tile_shared method with different block size.

- c. List the Op Times, whole program execution time, and accuracy for batch size of 100, 1k, and 5k images using this optimization (including any previous optimizations also used).

Blocksize 16:

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.376543 ms	1.47727 ms	0m0.213s	0.86
1000	3.57791 ms	14.5086 ms	0m0.280s	0.886
5000	17.7052 ms	72.5626 ms	0m0.960s	0.871

Blocksize 12:

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
5000	30.3936 ms	66.3541 ms	0m0.853s	0.871

Blocksize 8:

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
5000	56.8197 ms	167.527 ms	0m0.853s	0.871

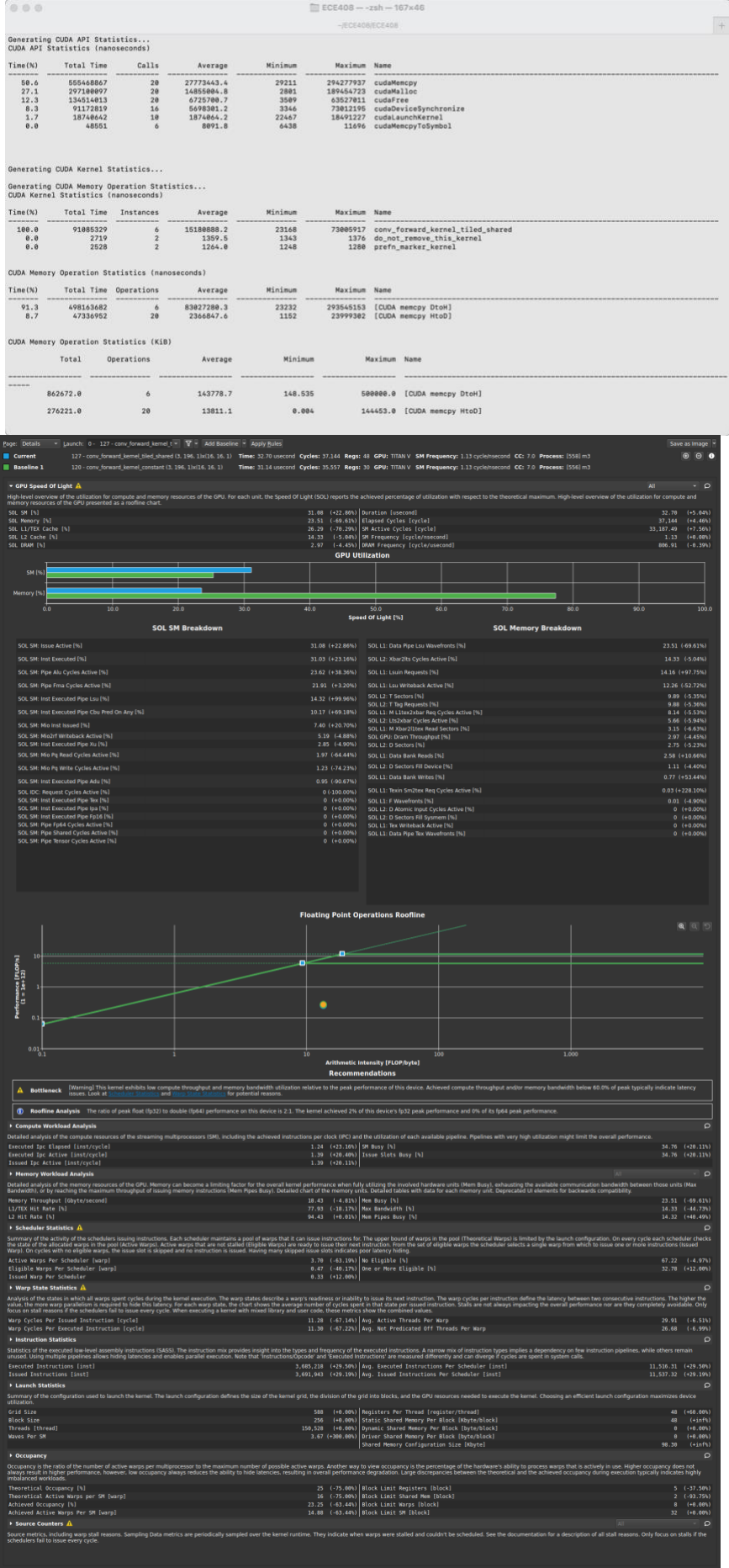
Blocksize 4:

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
5000	273.399 ms	558.748 ms	0m0.924s	0.871

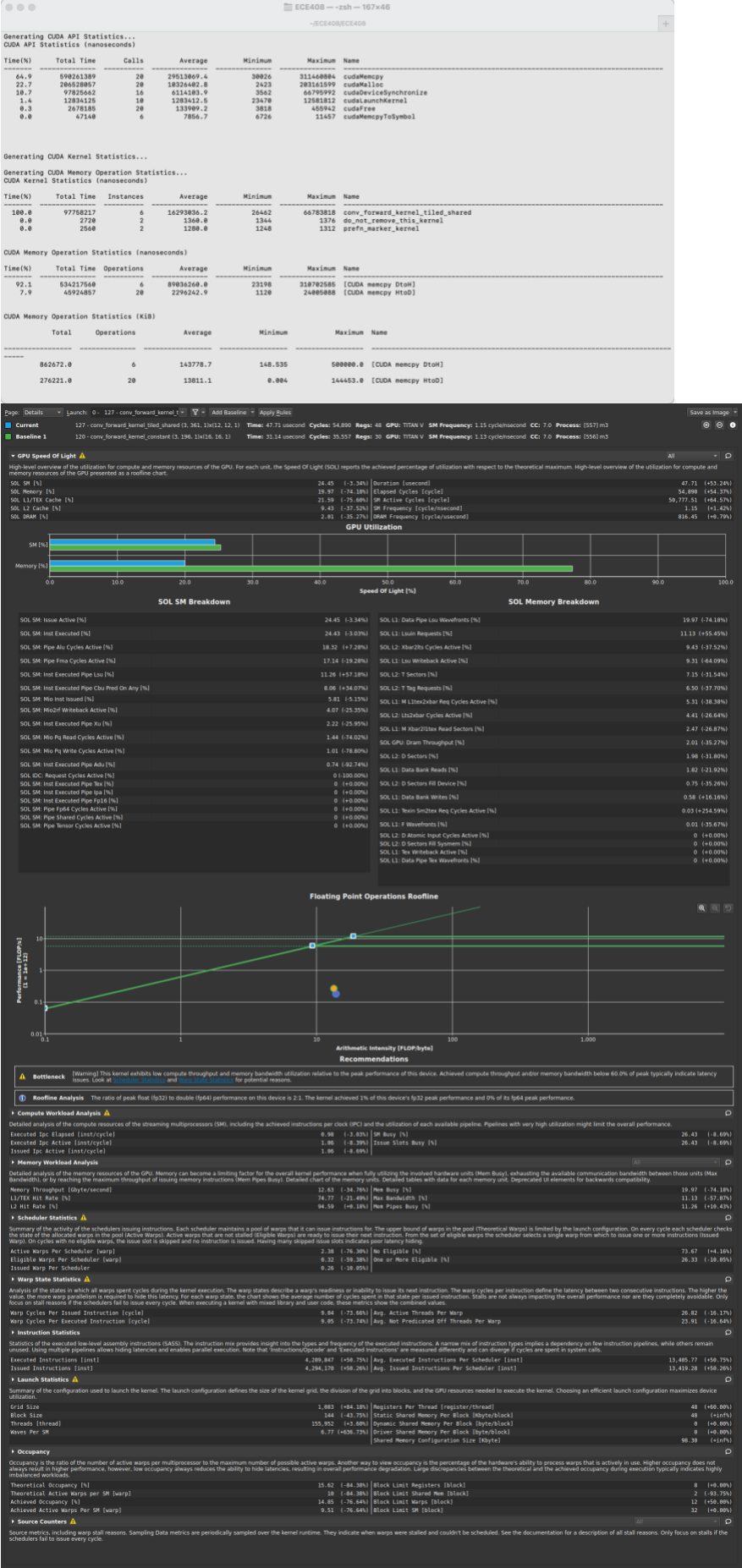
- d. Was implementing this optimization successful in improving performance? Why or why not? Include profiling results from *nsys* and *Nsight-Compute* to justify your answer, directly comparing to your baseline (or the previous optimization this one is built off of).

Yes. We can tell from the op time that it is the fastest when block size is 16 from the time of conv_forward_kernel_tiled_shared below. In this case, larger block means reducing more times reading from global memory because of preparing larger shared memory ahead. The opt time in the diagram above shows 12 is a better choice. I think it may relates with reading from the memory, and Blocksize 12 may fits better for coarsening. Or it can also accounts for the fluctuation, since it also has worse performance when I try to test multiple times. I only include the blocksize8, blocksize12 and blocksize16 nsys and nsight-compute here to justify my answer.

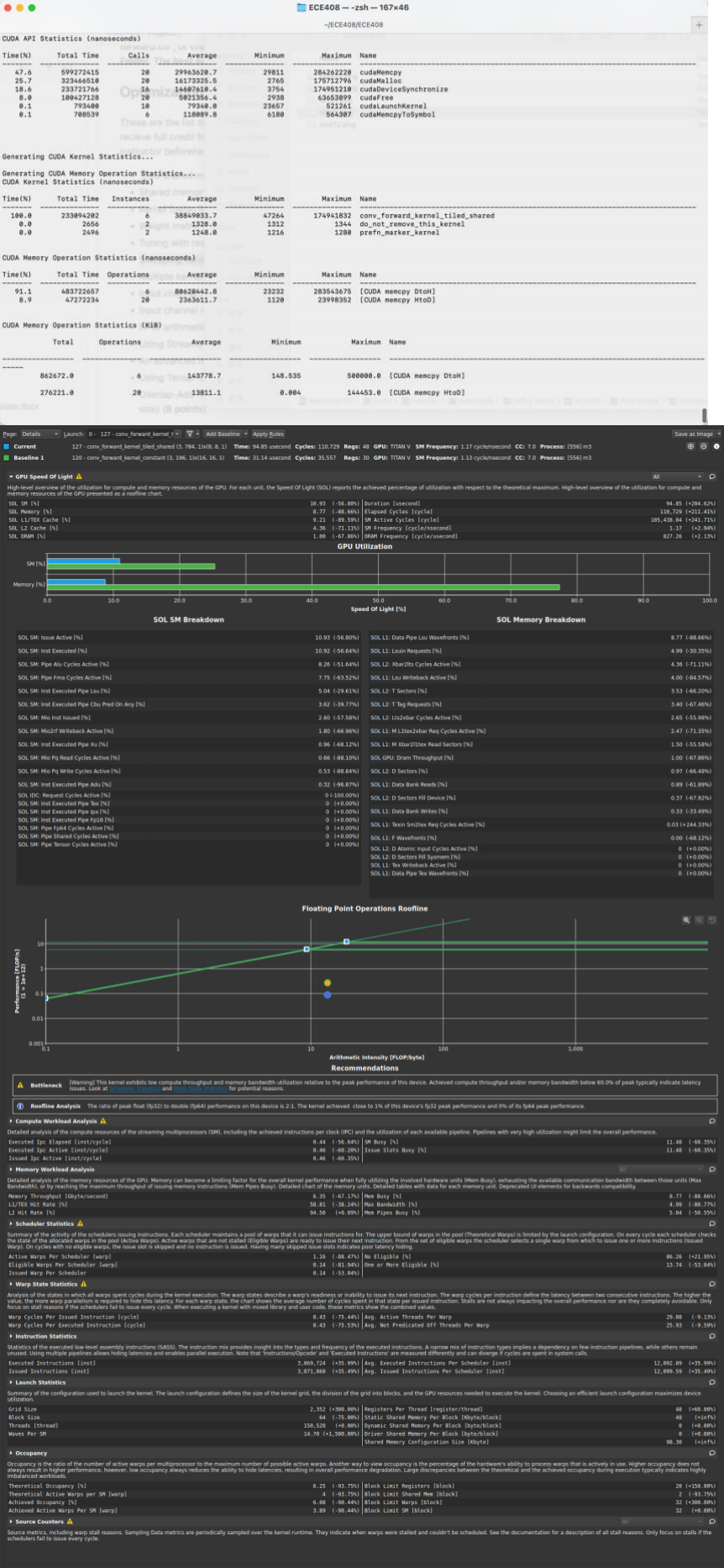
Blocksize16:



Blocksize12:



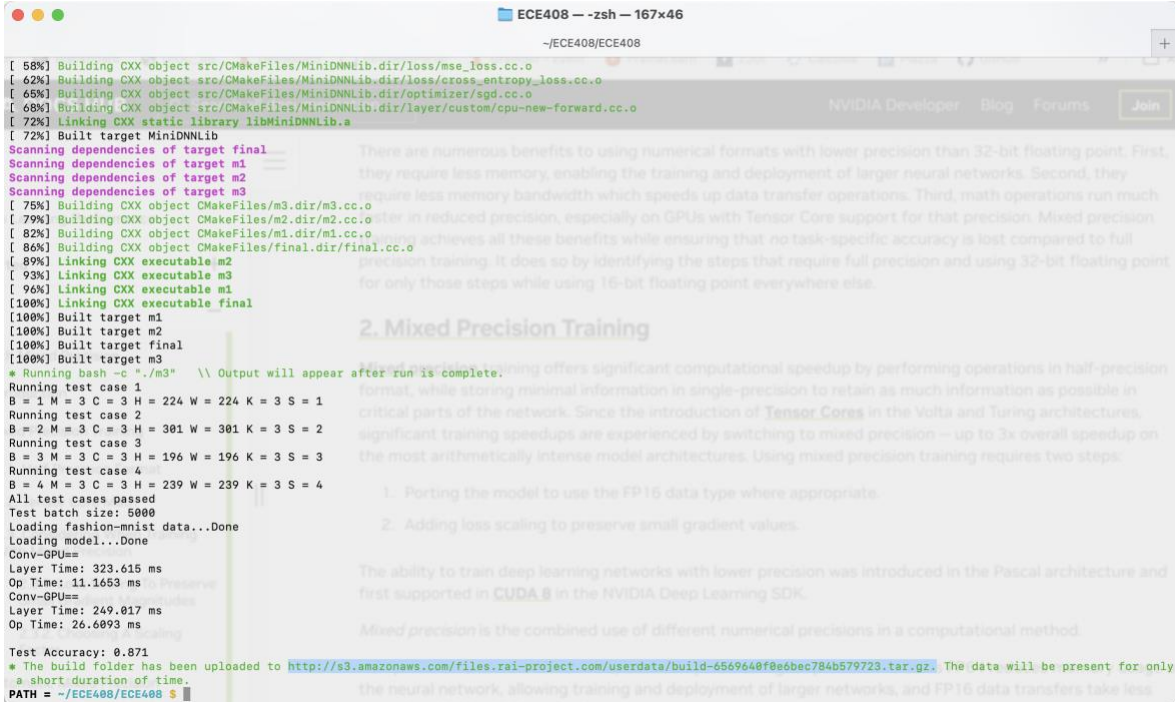
Blocksize8:



e. What references did you use when implementing this technique?

No reference

After adding all optimization into one part, I got the final result in 40ms.



```
[ 58%] Building CXX object src/CMakeFiles/MiniDNNLib.dir/loss/mse_loss.cc.o
[ 62%] Building CXX object src/CMakeFiles/MiniDNNLib.dir/loss/cross_entropy_loss.cc.o
[ 65%] Building CXX object src/CMakeFiles/MiniDNNLib.dir/optimizer/sgd.cc.o
[ 68%] Building CXX object src/CMakeFiles/MiniDNNLib.dir/layer/custom/cpu-new-forward.cc.o
[ 72%] Linking CXX static library libMiniDNNLib.a
[ 72%] Built target MiniDNNLib
Scanning dependencies of target final
Scanning dependencies of target m1
Scanning dependencies of target m2
Scanning dependencies of target m3
[ 75%] Building CXX object CMakeFiles/m3.dir/m3.cc.o
[ 79%] Building CXX object CMakeFiles/m2.dir/m2.cc.o
[ 82%] Building CXX object CMakeFiles/m1.dir/m1.cc.o
[ 86%] Building CXX object CMakeFiles/Final.dir/final.cc.o
[ 89%] Linking CXX executable m2
[ 93%] Linking CXX executable m3
[ 96%] Linking CXX executable m1
[100%] Linking CXX executable final
[100%] Built target m1
[100%] Built target m2
[100%] Built target final
[100%] Built target m3
* Running bash -c ". /m3" \\ Output will appear after run is complete.
Running test case 1
B = 1 M = 3 C = 3 H = 224 W = 224 K = 3 S = 1
Running test case 2
B = 2 M = 3 C = 3 H = 301 W = 301 K = 3 S = 2
Running test case 3
B = 3 M = 3 C = 3 H = 196 W = 196 K = 3 S = 3
Running test case 4
B = 4 M = 3 C = 3 H = 239 W = 239 K = 3 S = 4
All test cases passed
Test batch size: 5000
Loading fashion-mnist data...Done
Loading model...Done
Conv-GPU==
Layer Time: 323.615 ms
Op Time: 11.1653 ms
Conv-GPU==
Layer Time: 249.017 ms
Op Time: 26.6093 ms
Test Accuracy: 0.871
* The build folder has been uploaded to http://s3.amazonaws.com/files.ral-project.com/userdata/build-6569640f0e6bec784b579723.tar.gz. The data will be present for only a short duration of time.
PATH = ~/ECE408/ECE408 $
```

2. Mixed Precision Training

There are numerous benefits to using numerical formats with lower precision than 32-bit floating point. First, they require less memory, enabling the training and deployment of larger neural networks. Second, they require less memory bandwidth which speeds up data transfer operations. Third, math operations run much faster in reduced precision, especially on GPUs with Tensor Core support for that precision. Mixed precision achieves all these benefits while ensuring that no task-specific accuracy is lost compared to full precision training. It does so by identifying the steps that require full precision and using 32-bit floating point for only those steps while using 16-bit floating point everywhere else.

Mixed precision offers significant computational speedup by performing operations in half-precision format, while storing minimal information in single-precision to retain as much information as possible in critical parts of the network. Since the introduction of **Tensor Cores** in the Volta and Turing architectures, significant training speedups are experienced by switching to mixed precision – up to 3x overall speedup on the most arithmetically intense model architectures. Using mixed precision training requires two steps:

1. Porting the model to use the FP16 data type where appropriate.
2. Adding loss scaling to preserve small gradient values.

The ability to train deep learning networks with lower precision was introduced in the Pascal architecture and first supported in **CUDA 8** in the NVIDIA Deep Learning SDK.

Mixed precision is the combined use of different numerical precisions in a computational method.

The neural network, allowing training and deployment of larger networks, and FP16 data transfers take less